

DAA LAB FILE



SUBMITTED BY:-

NAME : SHAURYA PUNDIR

UNIV. ROLL NO : 2021975

CLASS ROLL NO : 56

SUBMITTED TO :-

Dr. Upma Jain

WEEK 1

/*Q1.Given an array of nonnegative integers, design a linear algorithm and implement it using a program to find whether given key element is present in the array or not. Also, find total number of comparisons for each input case. (Time Complexity = $O(n)$, where n is the size of input*/

```
#include<stdio.h>

int main()
{
    int n,num,flag=0,c=0;
    int arr[]={13,2,31,12,44,56,67,9,57,12,20,33,11,1};
    n = sizeof(arr)/sizeof(arr[0]);
    printf("\nEnter the roll number to be find:");
    scanf("%d",&num);
    for(int i=0;i<n;i++ )
    {
        c=c+1;
        if(arr[i]==num)
        {
            flag=1;
            printf("\nThe roll number u searched for is present.");
            break;
        }
    }
    if(flag==0)
    {
        printf("\nThe number u searched for is absent.");
    }
    printf("\nThe number of comparison done was:%d",c);
    return 0;
}
```

OUTPUT

Enter the roll number to be find: 44

The roll number u searched for is present.

The number of comparison done was: 5

/*Q2)Given an already sorted array of positive integers, design an algorithm and implement it using a program to find whether given key element is present in the array or not. Also, find total number of comparisons for each input case*/

```
#include <iostream>
#include <vector>
using namespace std;
pair<int, int> binarySearch(const vector<int>& array, int key) {
    int low = 0;
    int high = array.size() - 1;
    int comparisons = 0;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        comparisons++;
        if (array[mid] == key) {
            return make_pair(mid, comparisons);
        } else if (array[mid] < key) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }
    return make_pair(-1, comparisons);
}

int main() {
    vector<int> array = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10}; // Example sorted array
    int key = 5; // Example key to search
    pair<int, int> result = binarySearch(array, key);
    if (result.first != -1) {
        cout << "Key found at index " << result.first << ", comparisons: " << result.second << endl;
    } else {
        cout << "Key not found, comparisons: " << result.second << endl; }
    return 0;
}
```

OUTPUT

Key found at index 4, comparisons: 1

/*Q3)Given an already sorted array of positive integers, design an algorithm and implement it using a program to find whether a given key element is present in the sorted array or not. For an array arr[n], search at the indexes arr[0], arr[2], arr[4],.....,arr[2k] and so on. Once the interval (arr[2k] <key < arr[2k+1]) is found, perform a linear search operation from the index 2k to find the elementkey. (Complexity < O(n), where n is the number of elements need to be scanned for searching)*/

```
#include<stdio.h>
```

```
#include<math.h>
```

```
int main()
```

```
{    int num = 0, k = 0, low=0 ,flag = 0;
```

```
    int a[] = { 1,2,3,4,5,6,7,8,8,9,10,11,12,13,14,15,16,17};
```

```
    int size = sizeof(a) / sizeof(a[0]);
```

```
    printf("Enter the number to be searched: ");
```

```
    scanf("%d", &num);
```

```
    int s=sqrt(size);
```

```
    for (int i = 0; i < size; i += 2) {
```

```
        if (num==a[i])
```

```
        {
```

```
            flag=1;
```

```
            printf("\nNumber found.");
```

```
            break;
```

```
        }
```

```
        if(a[i]<num)
```

```
        { low=i;
```

```
        }
```

```
        if (a[i]>num) // If found, no need to continue searching
```

```
            break;
```

```
    }
```

```
    if(flag!=1)
```

```
{ for(int i=low;i<size;i++)
```

```
{
```

```
    if(a[i]==num)
```

```
    {
```

```
        printf("\nNumber found");
```

```
        flag=1;
```

```
    }  
    }  
}  
if(flag!=1)  
{  
    printf("\nNumber was not found");  
}  
return 0;  
}
```

OUTPUT

Enter the number to be searched: 8

Number found.

Week 2 :

/* Q1) Given a sorted array of positive integers containing few duplicate elements, design an algorithm and implement it using a program to find whether the given key element is present in the array or not. If present, then also find the number of copies of given key. (Time Complexity = $O(\log n)$)*/*

```
#include<stdio.h>
```

```
int main()
```

$$\{$$

```
int n,num,flag=0,c=0;
```

```
int arr[]={1,2,3,4,4,5,7,12,22,23,57,57,67};
```

```
n = sizeof(arr)/sizeof(arr[0]);
```

```
printf("\nEnter the number to be searched:");
```

scanf("%d",#);

```
if(num<arr[n/2])
```

 $\{$

```
for(int i=0;i<n/2;i++ )
```

 $\{$

```
if(arr[i]==num)
```

 $\{$

```
c=c+1;
```

```
if(c<1)
```

 $\{$

```
printf("\nThe number is present.");
```

$$\}$$
$$\}$$
$$\}$$

}

else

 $\{$

```
for(int i=n/2;i<n;i++ )
```

 $\{$

```
if(arr[i]==num)
```

$$\{$$

```
c=c+1;
```

```
if(c<1)
```

 $\{$

```
printf("\nThe number is present.");
```

```
    }  
    }  
}  
}  
if(c>0)  
{  
    printf("\nThe number of times its repeated is:%d",c);  
}  
else  
{  
    printf("\nNumber is not present.");  
}  
return 0;  
}
```

OUTPUT

Enter the number to be searched: 4

The number is present.

The number of times it's repeated is: 2

/*Q2) Given a sorted array of positive integers, design an algorithm and implement it using a program to find three indices i, j, k such that $arr[i] + arr[j] = arr[k]$.*/

```
#include <stdio.h>
```

```
int main() {
```

```
    int T;
```

```
    printf("Enter the number of test cases: ");
```

```
    scanf("%d", &T);
```

```
    for (int t = 0; t < T; t++) {
```

```
        int n;
```

```
        printf("Enter the size of the array: ");
```

```
        scanf("%d", &n);
```

```
        int arr[n];
```

```
        printf("Enter the elements of the array: ");
```

```
        for (int i = 0; i < n; i++) {
```

```
            scanf("%d", &arr[i]);
```

```
        }
```

```
        int found = 0;
```

```
        for (int i = 0; i < n - 2; i++) {
```

```
            for (int j = i + 1; j < n - 1; j++) {
```

```
                int sum = arr[i] + arr[j];
```

```
                for (int k = j + 1; k < n; k++) {
```

```
                    if (arr[k] == sum) {
```

```
                        printf("%d %d %d\n", i, j, k);
```

```
                        found = 1;
```

```
                        break;
```

```
                    }
```

```
                }
```

```
                if (found) break;
```

```
            }
```

```
            if (found) break;
```

```
        }
```

```
        if (!found) {
```

```
            printf("No sequence found\n");
```

```
        }
```

```
    }
```

```
    return 0;
```


}

OUTPUT

Enter the number of test cases: 1

Enter the size of the array: 5

Enter the elements of the array: 1 2 3 4 5

0 1 2

/*Q3Given an array of nonnegative integers, design an algorithm and a program to count the number of pairs of integers such that their difference is equal to a given key, K./*

```
#include<stdio.h>

#define max 100

int main()
{
    int arr[max],n,key;
    printf("\nEnter the value of n:");
    scanf("%d", &n);
    printf("\nEnter the values in array");
    for(int i=0;i<n;i++)
    {
        scanf("%d", &arr[i]);
    }
    printf("\nEnter the value of key:");
    scanf("%d", &key);
    for(int i=0;i<n;i++)
    {
        for(int j=i+1;j<n;j++)
        {
            if((arr[i]-arr[j]==key)||((arr[i]-arr[j]==(-key))))
            {
                printf("\n%d & %d form one such pair",arr[i],arr[j]);
            }
        }
    }
    return 0;
}
```

OUTPUT

Enter the value of n: 5

Enter the values in array: 1 5 3 4 2

Enter the value of key: 3

1 & 4 form one such pair

5 & 2 form one such pair

WEEK 3

/*Q1) given an unsorted array of integers, design an algorithm and a program to sort the array using insertion sort. Your program should be able to find number of comparisons and shifts (shifts -total number of times the array elements are shifted from their place) required for sorting the

```
array.*/

#include <iostream>

#include <vector>

using namespace std;

pair<int, int> insertionSort(vector<int>& arr) {

    int comparisons = 0;

    int shifts = 0;

    int n = arr.size();

    for (int i = 1; i < n; ++i) {

        int key = arr[i];

        int j = i - 1;

        while (j >= 0 && arr[j] > key) {

            ++comparisons;

            arr[j + 1] = arr[j];

            ++shifts;

            --j;

        }

        arr[j + 1] = key;

        ++shifts;

    }

    return make_pair(comparisons, shifts);

}

int main() {

    vector<int> arr = {12, 11, 13, 5, 6};

    pair<int, int> counts = insertionSort(arr);

    cout << "Sorted array:";

    for (int num : arr) {

        cout << " " << num;

    }

    cout << endl;

    cout << "Number of comparisons: " << counts.first << endl;

    cout << "Number of shifts: " << counts.second << endl;
```

```
    return 0;  
}
```

OUTPUT

Sorted array: 5 6 11 12 13

Number of comparisons: 7

Number of shifts: 12

/*Q2) I. Given an unsorted array of integers, design an algorithm and implement a program to sort this array using selection sort. Your program should also find number of comparisons and number of swaps required.*/

```
#include <iostream>

#include <vector>

using namespace std;

pair<int, int> selectionSort(vector<int>& arr) {

    int comparisons = 0;

    int swaps = 0;

    int n = arr.size();

    for (int i = 0; i < n - 1; ++i) {

        int min_idx = i;

        for (int j = i + 1; j < n; ++j) {

            ++comparisons;

            if (arr[j] < arr[min_idx]) {

                min_idx = j;

            }

        }

        if (min_idx != i) {

            swap(arr[i], arr[min_idx]);

            ++swaps;

        }

    }

    return make_pair(comparisons, swaps);

}

int main() {

    vector<int> arr = {12, 11, 13, 5, 6};

    pair<int, int> counts = selectionSort(arr);

    cout << "Sorted array:";

    for (int num : arr) {

        cout << " " << num;

    }

    cout << endl;

    cout << "Number of comparisons: " << counts.first << endl;

    cout << "Number of swaps: " << counts.second << endl;

    return 0;

}
```

OUTPUT

Sorted array: 5 6 11 12 13

Number of comparisons: 10

Number of swaps: 4

/*Q3) Given an unsorted array of positive integers, design an algorithm and implement it using a program to find whether there are any duplicate elements in the array or not./*

week 3:-q3#include <iostream>

#include <vector>

using namespace std;

int partition(int arr[], int low, int high) {

int pivot = arr[high];

int i = (low - 1);

for (int j = low; j < high; j++)

{

if (arr[j] < pivot) {

i++;

swap(arr[i], arr[j]);

}

}

swap(arr[i + 1], arr[high]);

return (i + 1);

}

void quickSort(int arr[], int low, int high) {

if (low < high) {

int pivotIndex = partition(arr, low, high);

quickSort(arr, low, pivotIndex - 1);

quickSort(arr, pivotIndex + 1, high);

}

}

int main() {

vector<int> a;

int n, x, count=0, j;

cout << "Enter the number of elements you want to enter: ";

cin >> n;

for (int i = 0; i < n; i++) {

cin >> x;

a.push_back(x);

}

quickSort(a.data(), 0, n - 1);

cout << "The sorted array is: ";

for (int i = 0; i < n; i++) {

```

        cout << a[i] << " ";
    }
    for(int i=0;i<n;i++)
    {
        for(j=i+1;j<n;j++)
        {
            if(a[i]==a[j])
            {
                count+=1;
            }
        }
        if((count>0) && (a[i]!=a[i-1]))
        {
            cout<<"\nThe duplicate element of "<<a[i]<<" is :"<<count+1;
        }
        count=0;
    }
    return 0;
}

```

OUTPUT

Enter the number of elements you want to enter: 5

12 11 13 5 6

The sorted array is: 5 6 11 12 13

WEEK 4

/*Q1)Given an unsorted array of integers, design an algorithm and implement it using a program to sort an array of elements by dividing the array into two subarrays and combining these subarrays after sorting each one of them. Your program should also find number of comparisons and Inversions during sorting the array.*/

```
#include <iostream>

using namespace std;

int inversion=0,comparison=0;

void merge(int a[], int l, int m, int r);

void mergesort(int a[], int l, int r)
{
    if (l < r) {
        int m = (l + r) / 2;
        mergesort(a, l, m);
        mergesort(a, m + 1, r);
        merge(a, l, m, r);
    }
}

void merge(int a[], int l, int m, int r)
{
    int n1 = m - l + 1;
    int n2 = r - m;
    int left[n1], right[n2];
    for (int i = 0; i < n1; i++)
        left[i] = a[l + i];
    for (int j = 0; j < n2; j++)
        right[j] = a[m + 1 + j];
    int i = 0, j = 0, k = l;
    while (i < n1 && j < n2)
    {
        comparison++;
        if (left[i] <= right[j])
        {
            a[k++] = left[i++];
        } else
        {
            inversion++;
            a[k++] = right[j++];
        }
    }
}
```

```

    }
}
while (i < n1) {
    a[k++] = left[i++];
}
while (j < n2)
{
    a[k++] = right[j++];
}
}
int main() {
    int n;
    cout << "Enter the value of n: ";
    cin >> n;
    int arr[n];
    cout << "Enter the elements of the array:\n";
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }
    mergesort(arr, 0, n - 1);
    cout << "Sorted array:\n";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
    cout<<"The number of comparison & inversion is: "<<comparison<<" "<<inversion;
    return 0;
}

```

OUTPUT

Enter the value of n: 5

Enter the elements of the array:

12 11 13 5 6

Sorted array:

5 6 11 12 13

The number of comparison & inversion is: 7 4

/*Q2)Given an unsorted array of integers, design an algorithm and implement it using a program to sort an array of elements by partitioning the array into two subarrays based on a pivot element such that one of the sub array holds values smaller than the pivot element while another sub array holds values greater than the pivot element. Pivot element should be selected randomly from the array. Your program should also find number of comparisons and swaps required for sorting the array.*/

```
#include <iostream>
using namespace std;
int comp=0,swaps=0;
void quicksort(int a[], int lb, int ub);
int pivot(int a[], int lb, int ub);
void quicksort(int a[], int lb, int ub)
{
    if (lb < ub)
    {
        int loc = pivot(a, lb, ub);
        quicksort(a, lb, loc - 1);
        quicksort(a, loc + 1, ub);
    }
}
int pivot(int a[], int lb, int ub)
{
    int pivot = a[lb];
    int start = lb, end = ub;
    while (start < end)
    {
        comp++;
        while (a[start] <= pivot)
        {
            start++;
        }
        while (a[end] > pivot)
        {
            end--;
        }
        if (start < end)
```

```

    {
        swap(a[start],a[end]);
        swaps++;
    }
} // Swap pivot element to its correct position
swap(a[lb], a[end]);
swaps++;
return end;
}
int main()
{
    int n;
    cout << "Enter the value of n: ";
    cin >> n;
    int arr[n];
    cout << "Enter the elements of the array:\n";
    for (int i = 0; i < n; i++)
    {
        cin >> arr[i];
    }
    quicksort(arr, 0, n - 1);
    cout << "Sorted array:\n";
    for (int i = 0; i < n; i++)
    { cout << arr[i] << " "; }
    cout << "Swaps & comparisons : "<<swaps<< " "<<comp<<endl;
    return 0;
}

```

OUTPUT

Enter the value of n: 5

Enter the elements of the array:

12 11 13 5 6

Sorted array:

5 6 11 12 13

Swaps & comparisons :5 7

/*Q3) . Given an unsorted array of integers, design an algorithm and implement it using a program to find Kth smallest or largest element in the array. */

```
#include <iostream>
```

```
using namespace std;
```

```
int partition(int arr[], int lb, int ub) {
```

```
    int pivot = arr[lb];
```

```
    int start = lb, end = ub;
```

```
    while (start < end) {
```

```
        while (arr[start] <= pivot) {
```

```
            start++;
```

```
        }
```

```
        while (arr[end] > pivot) {
```

```
            end--;
```

```
        }
```

```
        if (start < end) {
```

```
            swap(arr[start], arr[end]);
```

```
        }
```

```
    }
```

```
    swap(arr[lb], arr[end]);
```

```
    return end;
```

```
}
```

```
int quickselect(int arr[], int lb, int ub, int k) {
```

```
    if (lb <= ub) {
```

```
        int loc = partition(arr, lb, ub);
```

```
        if (loc == k) {
```

```
            return arr[loc];
```

```
        } else if (loc < k) {
```

```
            return quickselect(arr, loc + 1, ub, k);
```

```
        } else {
```

```
            return quickselect(arr, lb, loc - 1, k);
```

```
        }
```

```
    }
```

```
    return -1; // Indicates invalid input or k out of range
```

```
}
```

```

int main() {
    int n, k;
    cout << "Enter the value of n and k: ";
    cin >> n >> k;
    int arr[n];
    cout << "Enter the elements of the array:\n";
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }

    int kthSmallest = quickselect(arr, 0, n - 1, k - 1);
    int kthLargest = quickselect(arr, 0, n - 1, n - k);

    cout << "The kth smallest element is: " << kthSmallest << endl;
    cout << "The kth largest element is: " << kthLargest << endl;

    return 0;
}

```

OUTPUT

```

Enter the value of n and k: 6 3
Enter the elements of the array:
12 11 13 5 6 7
The kth smallest element is: 7
The kth largest element is: 12

```

WEEK 5

/*Q1) Given an unsorted array of alphabets containing duplicate elements. Design an algorithm and implement it using a program to find which alphabet has maximum number of occurrences and print it.*/

```
#include <iostream>

using namespace std;

void countsort(char a[], int n) {
    const int RANGE = 26; // Considering only alphabets
    int count[RANGE] = {0};
    for(int i = 0; i < n; i++)
    {
        count[a[i] - 'a']++;
    }
    char max_occurring;
    int max_count = 0;
    for(int i = 0; i < RANGE; i++) {
        if(count[i] > max_count) {
            max_count = count[i];
            max_occurring = 'a' + i;
        }
    }
    cout << "The element with max repetition: " << max_occurring << endl;
}

int main() {
    int max = 100;
    char arr[max];
    int n;
    cout << "Enter the value of n: ";
    cin >> n;
    cout << "Enter the value in array: ";
    for(int i = 0; i < n; i++) {
        cin >> arr[i];
    }
    countsort(arr, n);
    return 0;
}
```

OUTPUT

Enter the value of n: 8

Enter the value in array: a b c d e f f f

The element with max repetition: f

/*Q2) I. Given an unsorted array of integers, design an algorithm and implement it using a program to find whether two elements exist such that their sum is equal to the given key element. */

```
#include <iostream>

#include <algorithm>

using namespace std;

void findPair(int arr[], int n, int key) {

    sort(arr, arr + n); // Sort the array in O(n log n) time

    int left = 0;

    int right = n - 1;

    while (left < right) {

        int sum = arr[left] + arr[right];

        if (sum == key) {

            cout << "Pair found: " << arr[left] << " " << arr[right] << endl;

            return;

        } else if (sum < key) {

            left++;

        } else {

            right--;

        }

    }

    cout << "No such pair found." << endl;

}

int main() {

    int n, key;

    cout << "Enter the number of elements in the array: ";

    cin >> n;

    int arr[n];

    cout << "Enter the values: ";

    for(int i = 0; i < n; i++) {

        cin >> arr[i];

    }

    cout << "Enter the key element: ";

    cin >> key;
```

```
findPair(arr, n, key);  
return 0;  
}
```

OUTPUT

Enter the number of elements in the array: 6

Enter the values: 1 2 3 4 5 6

Enter the key element: 7

Pair found: 1 6

/*Q3) You have been given two sorted integer arrays of size m and n. Design an algorithm and implement it using a program to find list of elements which are common to both.*/

```
#include <iostream>

#include <vector>

using namespace std;

vector<int> findCommonElements(int arr1[], int m, int arr2[], int n) {

    vector<int> commonElements;

    int i = 0, j = 0;

    while (i < m && j < n) {

        if (arr1[i] == arr2[j]) {

            commonElements.push_back(arr1[i]);

            i++;

            j++;

        } else if (arr1[i] < arr2[j]) {

            i++;

        } else {

            j++;

        }

    }

    return commonElements;

}

int main() {

    int m, n;

    cout << "Enter the size of the first array: ";

    cin >> m;

    int arr1[m];

    cout << "Enter the elements of the first array (sorted): ";

    for (int i = 0; i < m; i++) {

        cin >> arr1[i];

    }

    cout << "Enter the size of the second array: ";

    cin >> n;

    int arr2[n];

    cout << "Enter the elements of the second array (sorted): ";

    for (int i = 0; i < n; i++) {

        cin >> arr2[i];

    }

}
```

```
}  
vector<int> commonElements = findCommonElements(arr1, m, arr2, n);  
cout << "Common elements: ";  
for (int i = 0; i < commonElements.size(); i++) {  
    cout << commonElements[i] << " ";  
}  
cout << endl;  
return 0;  
}
```

OUTPUT

Enter the size of the first array: 5

Enter the elements of the first array (sorted): 1 3 4 5 6

Enter the size of the second array: 4

Enter the elements of the second array (sorted): 2 3 5 7

Common elements: 3 5

WEEK 6

/*Q1) Given a (directed/undirected) graph, design an algorithm and implement it using a program to find if a path exists between two given vertices or not. (Hint: use DFS)*/

```
#include<iostream>

#include<vector>

using namespace std;

void dfs(int node,vector<int>adj[],int vis[],vector<int>&ls )
{
    vis[node]=1;
    ls.push_back(node);
    for(auto it:adj[node])
    {
        if(!vis[it])
        {
            dfs(it,adj,vis,ls);
        }
    }
}

vector<int> dfssofar(int source,int n,vector<int>adj[])
{
    int start=source;
    int vis[n+1]={0};
    vector<int>ls;
    dfs(start,adj,vis,ls);
    return ls;
}

int main() {
    int n, m,source,destination,flag=0;
    cout<<"\nEnter the number of nodes and number of edges: "<<endl;
    cin >> n >> m;
    vector<int> adj[n+1];
    for(int i = 0; i < m; i++) {
        int u, k;
        cin >> u >> k;
        adj[u].push_back(k);
    }
}
```

```

cout<<"\nEnter the source and destination nodes :"<<endl;
cin>>source>>destination;
vector<int>traversal=dfsofgraph(source,n,adj);
for(int i = 0; i < traversal.size(); i++)
{
    if(traversal[i]==destination)
    {
        cout<<"\nThe path exist.";
        flag=1;
        break;
    }
}
if(flag==0)
{
    cout<<"The path doesnt exist."<<endl;
}
return 0;
}

```

OUTPUT

Enter the number of nodes and number of edges:

5 4

1 2

2 3

3 4

4 5

Enter the source and destination nodes :

1 5

The path exist.

/*Q2) Given a graph, design an algorithm and implement it using a program to find if a graph is bipartite or not. (Hint: use BFS)*/

```
#include<iostream>

#include<vector>          //CHECK IF A GRAPH IS BARPATITE OR NOT , IF RAPH IS LINEAR SO ITS
BARPATITE

#include<queue>          //IF A GRAPH IS WITH EVEN CYCLE THEN BARPATITE ELSE NOT BARPATITE

using namespace std;

bool bfsearch(int n,vector<int>adj[]) {
    vector<int> bfs;
    queue<int> q;
    int vis[n+1]; // Change here to n+1
    for(int i=1; i<=n; i++) vis[i] = -1; // Change here, indexing starts from 1
    for(int i = 1; i <= n; ++i) {
        if(vis[i] == -1) { // Change here, we're checking for -1, not 0
            vis[i] = 1;
            q.push(i);
            while(!q.empty()) {
                int node = q.front();
                q.pop();
                bfs.push_back(node);
                for(auto it : adj[node]) {
                    if(vis[it] == -1) { // Change here, checking for -1
                        vis[it] = 1 - vis[node]; // Change here, flipping colors
                        q.push(it);
                    }
                    else if(vis[it] == vis[node]) {
                        return false;
                    }
                }
            }
        }
    }
    return true;
}

int main() {
    int n, m, start = 0;
```

```

cin >> n >> m;
vector<int> adj[n+1];
for(int i = 0; i < m; i++) {
    int u, k;
    cin >> u >> k;
    adj[u].push_back(k);
    adj[k].push_back(u); // Add this line to handle undirected edges
}
bool ans = bfsearch(n, adj);
if(ans == false) {
    cout << "Not a bipartite graph";
}
else {
    cout << "Bipartite graph";
}
return 0;
}

```

OUTPUT

```

5 4
1 2
2 3
3 4
4 5
Bipartite graph

```


/*Q3) Given a directed graph, design an algorithm and implement it using a program to find whether cycle exists in the graph or not.*/

```
#include<iostream>          //CYCLE DETECTION THROUGH BFS
```

```
#include<vector>
```

```
#include<queue>
```

```
using namespace std;
```

```
bool bfsearch(int n,vector<int>adj[]) {
```

```
    vector<int> bfs;
```

```
    queue<int> q;
```

```
    int vis[n+1]={0};
```

```
    vis[1] = 1;
```

```
    q.push(1);
```

```
    while(!q.empty()) {
```

```
        int node = q.front();
```

```
        q.pop();
```

```
        bfs.push_back(node);
```

```
        for(auto it : adj[node]) {
```

```
            if(vis[it] == 0) {
```

```
                vis[it] = 1;
```

```
                q.push(it);
```

```
            }
```

```
            else if(vis[it] == vis[node]) {
```

```
                return true; // Cycle detected
```

```
            }
```

```
        }
```

```
    }
```

```
    return false; // No cycle
```

```
}
```

```
int main() {
```

```
    int n, m, start = 0;
```

```
    cin >> n >> m;
```

```
    vector<int> adj[n+1];
```

```
    for(int i = 0; i < m; i++) {
```

```
        int u, k;
```

```
        cin >> u >> k;
```

```
adj[u].push_back(k);
adj[k].push_back(u); // Add this line to handle undirected edges
}
bool ans = bfsearch(n, adj);
if(ans == false) {
    cout << "Not a Cycle";
}
else {
    cout << "Cycle detected";
}
return 0;
}
```

OUTPUT

5 5

1 2

2 3

3 4

4 5

5 1

Cycle detected

WEEK 7

/*Q1)After end term examination, Akshay wants to party with his friends. All his friends are living as paying guest and it has been decided to first gather at Akshay's house and then move towards party location. The problem is that no one knows the exact address of his house in the city. Akshay as a computer science wizard knows how to apply his theory subjects in his real life and came up with an amazing idea to help his friends. He draws a graph by looking in to location of his house and his friends' location (as a node in the graph) on a map. He wishes to find out shortest distance and path covering that distance from each of his friend's location to his house and then whatsapp them this path so that they can reach his house in minimum time. Akshay has developed the program that implements Dijkstra's algorithm but not sure about correctness of results. Can you also implement the same algorithm and verify the correctness of Akshay's results? */

```
#include <bits/stdc++.h>

using namespace std;

class Solution
{
public:
    vector<int> dijkstra(int V, vector<vector<int>>> adj[], int S)
    {
        priority_queue<pair<int, int>, vector<pair<int, int>>, greater<pair<int, int>>> pq;
        vector<int> distTo(V, INT_MAX);
        distTo[S] = 0;
        pq.push({0, S});
        while (!pq.empty())
        {
            int node = pq.top().second;
            int dis = pq.top().first;
            pq.pop();
            for (auto it : adj[node])
            {
                int v = it[0];
                int w = it[1];
                if (dis + w < distTo[v])
                {
                    distTo[v] = dis + w;
                    pq.push({dis + w, v});
                }
            }
        }
        return distTo;
    }
}
```

```

};

int main()
{
    int n,e;
    cin>>n>>e;
    vector<vector<int>> adj[n];
    for(int i=0;i<e;i++)
    {
        int u,v,w;
        cin>>u>>v>>w;
        adj[u].push_back({v, w});
        adj[v].push_back({u, w});
    }
    Solution obj;
    for (int i = 0; i < n; i++)
    {
        vector<int> res = obj.dijkstra(n, adj, i);
        cout << "From node " << i << " to all other nodes: ";
        for (int j = 0; j < n; j++)
        { cout << res[j] << " "; }
        cout << endl;
    }
    return 0;
}

```

OUTPUT

```

4 4
0 1 5
1 2 2
2 3 3
0 3 9
From node 0 to all other nodes: 0 5 7 9
From node 1 to all other nodes: 5 0 2 5
From node 2 to all other nodes: 7 2 0 3
From node 3 to all other nodes: 9 5 3 0

```

/*Q2)Design an algorithm and implement it using a program to solve previous question's problem using Bellman- Ford's shortest path algorithm.*/

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
class Solution {
```

```
public:
```

```
    vector<int> bellman_ford(int V, vector<vector<int>>>adj, int S) {
```

```
        vector<int> dist(V,1e8);
```

```
        dist[S] = 0;
```

```
        for (int i = 0; i < V - 1; i++) {
```

```
            for (auto &it : adj) {
```

```
                int u = it[0];
```

```
                int v = it[1];
```

```
                int wt = it[2];
```

```
                if (dist[u] != 1e8 && dist[u] + wt < dist[v]) {
```

```
                    dist[v] = dist[u] + wt;
```

```
                }
```

```
            }
```

```
        }
```

```
        return dist;
```

```
    }
```

```
};
```

```
int main() {
```

```
    int n,e;
```

```
    cin>>n>>e;
```

```
    vector<vector<int>>> adj;
```

```
    for(int i=0;i<e;i++)
```

```
    {
```

```
        int u,v,w;
```

```
        cin>>u>>v>>w;
```

```
        adj.push_back({u,v, w});
```

```
    }
```

```
    int S = 0;
```

```
    Solution obj;
```

```
    for(int i=0;i<n;i++)
```

```
    {
```

```

vector<int> dist = obj.bellman_ford(n,adj,i);

    for (auto d : dist) {

        cout << d << " ";

    }

    cout << endl;

}

return 0;

}

```

OUTPUT

```

4 4
0 1 5
1 2 2
2 3 3
0 3 9
0 5 7 9
2147483647 0 2 2147483647
2147483647 2147483647 0 3
2147483647 2147483647 2147483647 0

```

/*Q3) Given a directed graph with two vertices (source and destination). Design an algorithm and implement it using a program to find the weight of the shortest path from source to destination with exactly k edges on the path.*/

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
#define INF INT_MAX
```

```
int shortestPath(vector<vector<int>>& graph, int u, int v, int k) {  
    int V = graph.size();  
    vector<vector<vector<int>>> dp(V, vector<vector<int>>(V, vector<int>(k+1, INF)));  
  
    for (int e = 0; e <= k; e++) {  
        for (int i = 0; i < V; i++) {  
            for (int j = 0; j < V; j++) {  
                if (e == 0 && i == j)  
                    dp[i][j][e] = 0;  
                if (e == 1 && graph[i][j] != INF)  
                    dp[i][j][e] = graph[i][j];  
                if (e > 1) {  
                    for (int a = 0; a < V; a++) {  
                        if (graph[i][a] != INF && i != a && j != a && dp[a][j][e-1] != INF)  
                            dp[i][j][e] = min(dp[i][j][e], graph[i][a] + dp[a][j][e-1]);  
                    }  
                }  
            }  
        }  
    }  
    return dp[u][v][k];  
}
```

```
int main() {
```

```
    int n,e;        //n is no of nodes int the array
```

```
    cin>>n>>e;
```

```
    vector<vector<int>> matrix(n, vector<int>(n, -1));
```

```
    for(int k = 0; k < e; k++) {
```

```
        int i,j,wt;
```

```
        cin>>i>>j>>wt;
```

```
        matrix[i][j] =wt;
```

```

    }

    /*int V = 4;

    vector<vector<int>> matrix(V, vector<int>(V, -1));

    matrix[0][1] = 2;
    matrix[1][0] = 1;
    matrix[1][2] = 3;
    matrix[3][0] = 3;
    matrix[3][1] = 5;
    matrix[3][2] = 4;*/

    int start,dest,k;

    cin>>start>>dest>>k;

    shortestPath(matrix,start,dest,k);

    for (auto row : matrix) {
        for (auto cell : row) {
            cout << cell << " ";
        }
        cout << endl;
    }

    return 0;
}

```

OUTPUT

```

4 4
0 1 2
1 2 3
3 0 3
3 1 5
0 3 2
0 2 4 -1
-1 0 3 -1
3 5 0 -1
2 4 -1 0

```


WEEK 8

/*Q1) Assume that a project of road construction to connect some cities is given to your friend. Map of these cities and roads which will connect them (after construction) is provided to him in the form of a graph. Certain amount of rupees is associated with construction of each road. Your friend has to calculate the minimum budget required for this project. The budget should be designed in such a way that the cost of connecting the cities should be minimum and number of roads required to connect all the cities should be minimum (if there are N cities then only N-1 roads need to be constructed). He asks you for help. Now, you have to help your friend by designing an algorithm which will find minimum cost required to connect these cities.*/

```
#include <bits/stdc++.h>

using namespace std;

class Solution
{
public:
    //Function to find sum of weights of edges of the Minimum Spanning Tree.
    int spanningTree(int V, vector<vector<int>> adj[])
    {
        priority_queue<pair<int, int>,
            vector<pair<int, int> >, greater<pair<int, int>>> pq; //priority_queue<pair<int, int>,
            vector<pair<int, int>>, less<pair<int, int>>> pq;

        vector<int> vis(V, 0);
        vector<pair<int, int>> mstEdges;
        // {wt, node}
        pq.push({0, 0});
        int sum = 0;
        while (!pq.empty()) {
            auto it = pq.top();
            pq.pop();
            int node = it.second;
            int wt = it.first;

            if (vis[node] == 1) continue;
            // add it to the mst
            vis[node] = 1;
            sum += wt;
            for (auto it : adj[node]) {
                int adjNode = it[0];
                int edW = it[1];
                if (!vis[adjNode])
```

```

        {
            pq.push({edW, adjNode});
            mstEdges.push_back({node, adjNode});
        }
    }

    return sum;
}

};

int main() {
    int n,e;

    cin>>n>>e;
    vector<vector<int>> adj[n];
    for(int i=0;i<e;i++)
    {
        int u,v,w;
        cin>>u>>v>>w;
        adj[u].push_back({v, w});
        adj[v].push_back({u, w});
    }

    Solution obj;
    int sum = obj.spanningTree(n, adj);
    cout << "The sum of all the edge weights: " << sum << endl;

    return 0;
}

```

OUTPUT

```

4 5
0 1 10
0 2 6
0 3 5
1 3 15
2 3 4

```

The sum of all the edge weights: 19

/*Q2) Implement the previous problem using Kruskal's algorithm.*/

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
class DisjointSet {
```

```
public:
```

```
    vector<int> rank, parent;
```

```
    DisjointSet(int n) {
```

```
        rank.resize(n+1, 0);
```

```
        parent.resize(n+1);
```

```
        for(int i = 0; i <= n; i++) {
```

```
            parent[i] = i;
```

```
            // rank[i] = 0;
```

```
        }
```

```
    }
```

```
    int findUPar(int node) {
```

```
        if(node == parent[node])
```

```
            return node;
```

```
        return findUPar(parent[node]);
```

```
    }
```

```
    void unionByRank(int u, int v) {
```

```
        int ulp_u = findUPar(u);
```

```
        int ulp_v = findUPar(v);
```

```
        if(ulp_u == ulp_v) return;
```

```
        if(rank[ulp_u] < rank[ulp_v]) {
```

```
            parent[ulp_u] = ulp_v;
```

```
        }
```

```
        else if(rank[ulp_v] < rank[ulp_u]) {
```

```
            parent[ulp_v] = ulp_u;
```

```
        }
```

```
        else {
```

```
            parent[ulp_v] = ulp_u;
```

```
            rank[ulp_u]++;
```

```

    }
}

void disp()
{
    cout<<"\nparent"<<endl;
    for(int i : parent)
    {
        cout<<" "<<i;
    }
    cout<<"\nrank\n";
    for(int i: rank)
    {
        cout<<" "<<i;
    }
}
};

```

```

class Solution
{
public:
    int spanningTree(int V, vector<pair<int, int>> adj[])
    {
        vector<pair<int, pair<int, int>>> edges;
        for (int i = 0; i < V; i++) {
            for (auto it : adj[i]) {
                int adjNode = it.first;
                int wt = it.second;
                int node = i;

                edges.push_back({wt, {node, adjNode}});
            }
        }

        DisjointSet ds(V);
        sort(edges.begin(), edges.end());
        int mstWt = 0;
        for (auto it : edges) {

```

```

        int wt = it.first;

        int u = it.second.first;
        int v = it.second.second;

        if (ds.findUPar(u) != ds.findUPar(v)) {
            mstWt += wt;
            ds.unionByRank(u, v);
        }
    }
    ds.disp();
    return mstWt;
}

};

int main() {
    int n,e;
    cin>>n>>e;
    vector<pair<int, int>> adj[n];
    for(int i=0;i<e;i++)
    {
        int u,v,w;
        cin>>u>>v>>w;
        adj[u].push_back({v, w});
        adj[v].push_back({u, w});
    }
    Solution sol;
    int mstWeight = sol.spanningTree(n, adj);
    cout << "Weight of Minimum Spanning Tree: " << mstWeight << endl;

    return 0;
}

```

OUTPUT

parent

0 1 2 3

rank

0 0 0 0

Weight of Minimum Spanning Tree: 19

/*Q3) Assume that same road construction project is given to another person. The amount he will earn from this project is directly proportional to the budget of the project. This person is greedy, so he decided to maximize the budget by constructing those roads who have highest construction cost. Design an algorithm and implement it using a program to find the maximum budget required for the project.*/

```
#include <bits/stdc++.h>

using namespace std;

class Solution
{
public:
    //Function to find sum of weights of edges of the Minimum Spanning Tree.
    int spanningTree(int V, vector<vector<int>> adj[])
    {
        priority_queue<pair<int, int>, vector<pair<int, int>>, less<pair<int, int>>> pq;
        vector<int> vis(V, 0);
        vector<pair<int, int>> mstEdges;
        // {wt, node}
        pq.push({0, 0});
        int sum = 0;
        while (!pq.empty()) {
            auto it = pq.top();
            pq.pop();
            int node = it.second;
            int wt = it.first;

            if (vis[node] == 1) continue;
            // add it to the mst
            vis[node] = 1;
            sum += wt;
            for (auto it : adj[node]) {
                int adjNode = it[0];
                int edW = it[1];
                if (!vis[adjNode])
                {
                    pq.push({edW, adjNode});
                    mstEdges.push_back({node, adjNode});
                }
            }
        }
    }
}
```

```

        }
        return sum;
    }
};

int main() {
    int n,e;
    cin>>n>>e;
    vector<vector<int>> adj[n];
    for(int i=0;i<e;i++)
    {
        int u,v,w;
        cin>>u>>v>>w;
        adj[u].push_back({v, w});
        adj[v].push_back({u, w});
    }

```

```

        Solution obj;
        int sum = obj.spanningTree(n, adj);
        cout << "The sum of all the edge weights: " << sum << endl;
        return 0;
    }

```

OUTPUT

Input:

```

4 5
0 1 10
0 2 6
0 3 5
1 3 15
2 3 4

```

Output:

The sum of all the edge weights: 19

WEEK 9

/*Q1) Given a graph, Design an algorithm and implement it using a program to implement Floyd Warshall all pair shortest path algorithm. */

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
class Solution {
```

```
public:
```

```
    void shortest_distance(vector<vector<int>>&matrix) {
```

```
        int n = matrix.size();
```

```
        for (int i = 0; i < n; i++) {
```

```
            for (int j = 0; j < n; j++) {
```

```
                if (matrix[i][j] == -1) {
```

```
                    matrix[i][j] = 1e9;
```

```
                }
```

```
                if (i == j) matrix[i][j] = 0;
```

```
            }
```

```
        }
```

```
        for (int k = 0; k < n; k++) {
```

```
            for (int i = 0; i < n; i++) {
```

```
                for (int j = 0; j < n; j++) {
```

```
                    matrix[i][j] = min(matrix[i][j],
```

```
                        matrix[i][k] + matrix[k][j]);
```

```
                }
```

```
            }
```

```
        }
```

```
        for (int i = 0; i < n; i++) {
```

```
            for (int j = 0; j < n; j++) {
```

```
                if (matrix[i][j] == 1e9) {
```

```
                    matrix[i][j] = -1;
```

```
                }
```

```
            }
```

```
        }
```

```
    }
```

```
};
```

```
int main() {
```



```

int n,e;      //n is no of nodes int the array

cin>>n>>e;

vector<vector<int>> matrix(n, vector<int>(n, -1));

for(int k = 0; k < e; k++) {
int i,j,wt;

    cin>>i>>j>>wt;

    matrix[i][j] =wt;

}

Solution obj;

obj.shortest_distance(matrix);


for (auto row : matrix) {
    for (auto cell : row) {
        cout << cell << " ";
    }
    cout << endl;
}

return 0;
}

```

OUTPUT

Input:

```

4 5
0 1 10
0 2 6
0 3 5
1 3 15
2 3 4

```

Output:

```

0 10 6 5
10 0 16 15
6 16 0 4
5 15 4 0

```

/*Q2) . Given a knapsack of maximum capacity w. N items are provided, each having its own value and weight. You have to Design an algorithm and implement it using a program to find the list of the selected items such that the final selected content has weight w and has maximum value. You can take fractions of items,i.e. the items can be broken into smaller pieces so that you have to carry only a fraction x_i of item i, where $0 \leq x_i \leq 1$ */

```
#include<iostream>

#include<vector>

#include<algorithm>

using namespace std;

int main()
{
    vector<pair<float, pair<int, int>>>> items;
    vector<pair<int,int>>>ans;

    int n, k;
    float profit = 0, knapwt = 0;
    cin >> n;
    for(int i = 0; i < n; i++)
    {
        float wt, value;
        cin >> wt >> value;
        items.push_back({value / wt, {wt, value}});
    }
    cin >> k;
    sort(items.rbegin(), items.rend()); // Sort in descending order
    for(auto& it : items)
    {
        if((knapwt + it.second.first) <= k)
        {
            knapwt += it.second.first;
            profit += it.second.second;
            ans.push_back({it.second.second,it.second.first});
        }
        else if(knapwt < k)
        {
            float temp = k - knapwt; // Remaining weight in the knapsack
            knapwt += temp;
```

```

        profit += it.first * temp;
        ans.push_back({it.first*temp,temp});
        break;
    }
}
cout << "Maximum value : " << profit << endl;
cout << "item - weight" << endl;
for(auto& it: ans)
{
    cout << it.first << "-" << it.second << endl;
}
return 0;
}

```

OUTPUT

```

5
5 30
20 100
40 160
30 90
10 20
50
Maximum value : 230
item - weight
30-5
100-20
100-25

```

/*Q3) Given an array of elements. Assume arr[i] represents the size of file i. Write an algorithm and a program to merge all these files into single file with minimum computation. For given two files A and B with sizes m and n, computation cost of merging them is $O(m+n)$. (Hint: use greedy approach) */

```
#include <iostream>

#include <vector>

#include <algorithm>

using namespace std;

int main() {

    int n;

    cin >> n;

    vector<int> arr(n);

    for (int i = 0; i < n; ++i) {

        cin >> arr[i];

    }

    // Sort the array in non-decreasing order

    sort(arr.begin(), arr.end());

    int total_cost = 0;

    while (arr.size() > 1) {

        // Merge the first two elements

        int merged_size = arr[0] + arr[1];

        // Add the computation cost of merging these two elements

        total_cost += merged_size;

        // Remove the first two elements and insert the merged size

        arr.erase(arr.begin(), arr.begin() + 2);

        arr.insert(lower_bound(arr.begin(), arr.end(), merged_size), merged_size);

    }

    cout << "Minimum computation cost required: " << total_cost << endl;

    return 0;

}
```

OUTPUT

5

3 2 5 4 1

Minimum computation cost required: 33

WEEK 10

/*Q1) Given a list of activities with their starting time and finishing time. Your goal is to select maximum number of activities that can be performed by a single person such that selected activities must be non-conflicting. Any activity is said to be non-conflicting if starting time of an activity is greater than or equal to the finishing time of the other activity. Assume that a person can only work on a single activity at a time.*/

```
#include<bits/stdc++.h>

using namespace std;

typedef struct E
{
    int start,end;
}e;

bool compare(const e& a, const e& b)
{
    return a.end < b.end;
}

int main()
{
    vector<int>ans;
    int n;
    cin>>n;
    e arr[n];
    for(int i=0;i<n;i++)
    {
        cin>>arr[i].start;
    }
    for(int i=0;i<n;i++)
    {
        cin>>arr[i].end;
    }
    int endd=-1,count=0;
    sort(arr,arr+n,compare);
    for(int i=0;i<n;i++)
    {
        if(arr[i].start>endd)
        {
```

```
        count++;
        endd=arr[i].end;
        ans.push_back(i+1);
    }
}
cout<<"Maximum activities:"<<count<<endl;
cout<<"Seclected Tasks: ";
for(auto it: ans)
    cout<<it<<" ";
}
```

OUTPUT

```
5
1 3 0 5 8
2 4 6 7 9
Maximum activities: 4
Seclected Tasks: 1 2 4 5
```

/*Q2) Given a long list of tasks. Each task takes specific time to accomplish it and each task has a deadline associated with it. You have to design an algorithm and implement it using a program to find maximum number of tasks that can be completed without crossing their deadlines and also find list of selected tasks.*/

```
#include<bits/stdc++.h>

using namespace std;

struct Task {

    int time, deadline, id;

};

bool compare(Task a, Task b) {

    return a.deadline < b.deadline;

}

int main() {

    int n;

    cin >> n;

    vector<Task> tasks(n);

    for(int i = 0; i < n; i++) {

        cin >> tasks[i].time;

        tasks[i].id = i + 1;

    }

    for(int i = 0; i < n; i++) {

        cin >> tasks[i].deadline;

    }

    sort(tasks.begin(), tasks.end(), compare);

    vector<int> selectedTasks;

    int currentTime = 0;

    for(int i = 0; i < n; i++) {

        if(currentTime + tasks[i].time <= tasks[i].deadline) {

            currentTime += tasks[i].time;

            selectedTasks.push_back(tasks[i].id);

        }

    }

    cout << "Max number of tasks = " << selectedTasks.size() << endl;

    cout << "Selected task numbers : ";

    for(int i = 0; i < selectedTasks.size(); i++) {

        cout << selectedTasks[i] << " ";

    }

    return 0;}
```

OUTPUT

5

2 1 3 2 5

2 4 4 7 8

Max number of tasks = 3

Selected task numbers : 1 2 4

/*Q3) Given an unsorted array of elements, design an algorithm and implement it using a program to find whether majority element exists or not. Also find median of the array. A majority element is an element that appears more than $n/2$ times, where n is the size of array*/

```
#include<bits/stdc++.h>

#include<map>

using namespace std;

int main()
{
    int n;
    cin>>n;
    vector<int>v(n);
    map<int,int>m; //{v,freq}
    for(int i=0; i<n; i++)
    {
        cin>>v[i];
    }
    for(int i =0;i<n;i++)
    {
        m[v[i]]++;
    }
    for(int i=0;i<=n;i++)
    {
        if(m[i]>n/2)
        {
            cout<<"yes"<<endl;
            cout<<i<<endl;
        }
    }
    return 0;
}
```

OUTPUT

Enter the size of the array: 7

Enter the elements of the array: 3 1 3 3 2 3 3

Majority element exists.

Median of the array: 3

WEEK 11

/*Q1) Given a sequence of matrices, write an algorithm to find most efficient way to multiply these matrices together. To find the optimal solution, you need to find the order in which these matrices should be multiplied. */

```
#include<iostream>

#include<bits/stdc++.h>

using namespace std;

//using tabulation
int f(int i,int j,vector<int>&arr)
{
    int n=arr.size();
    int dp[n][n];
    for(int i=0;i<arr.size();i++)
        dp[i][i]=0;

    for(int i=n-1 ;i>=1 ;i--)
    {
        for(int j=i+1;j<n;j++)
        {
            int mini=1e9;
            for(int k=i;k<j;k++)
            {
                int steps=arr[i-1]*arr[k]*arr[j]+dp[i][k]+dp[k+1][j];
                mini=min(mini,steps);
            }
            dp[i][j] = mini;
        }
    }
    return dp[1][n-1];
}
```

```
int main()
{
    int n;
    cin >> n;
    vector<vector<int>>>dp(n,vector<int>(n,-1));
    vector<int> arr(n);
```

```
for (int i = 0; i < n; i++)
{
    cin >> arr[i];
}
// int s = f(1, n - 1, arr,dp); //for recursion and the momoisation
int s=f(1 ,n-1,arr);    //for tabulation
cout << s << endl;
return 0;
}
```

OUTPUT

Enter the number of matrices: 4

Enter the dimensions of matrices: 5 4 6 2 7

Minimum number of scalar multiplications required: 124

/*Q2) Given a set of available types of coins. Let suppose you have infinite supply of each type of coin. For a given value N, you have to Design an algorithm and implement it using a program to find number of ways in which these coins can be added to make sum value equals to N.*/

```
#include<iostream>

#include<bits/stdc++.h>

using namespace std;

//memoisation

int f(int n,vector<int>& coin,int target,vector<vector<int>>&dp)
{
    if(n==0)
    {
        if(target%coin[0]==0)
            return 1;
        return 0;
    }
    if(dp[n][target]!=-1) return dp[n][target];
    int notTake=f(n-1,coin,target,dp);
    int take=0;
    if(coin[n]<=target)
    {
        take=f(n,coin,target-coin[n],dp);
    }
    return dp[n][target]=take+notTake;
}

int main()
{
    int n;
    cin>>n;
    vector<vector<int>>dp(n,vector<int>(n,0));
    vector<int> coins(n,0);
    for(int i=0;i<n;i++)
    {
        cin>>coins[i];
    }
    int target;
    cin>>target;
```

```
// int ans=f(n,coins,target); //for recursion  
int ans=f(n,coins,target,dp); //for memoisation  
  
cout<<"The total no of ways to spend the coins is :"<<ans<<endl;  
}
```

OUTPUT

Enter the number of coins: 3

Enter the denominations of coins: 1 2 5

Enter the target amount: 5

The total number of ways to spend the coins is: 4

/*Q3) Given a set of elements, you have to partition the set into two subsets such that the sum of elements in both subsets is same. Design an algorithm and implement it using a program to solve this problem.*/

```
#include <iostream>

#include <vector>

using namespace std;

bool canPartition(vector<int>& nums) {
    int totalSum = 0;

    for (int num : nums) {
        totalSum += num;
    }

    if (totalSum % 2 != 0) {
        return false; // If sum is odd, cannot partition into two equal sum subsets
    }

    int targetSum = totalSum / 2;
    vector<bool> dp(targetSum + 1, false);
    dp[0] = true;

    for (int num : nums) {
        for (int j = targetSum; j >= num; j--) {
            dp[j] = dp[j] || dp[j - num];
        }
    }

    return dp[targetSum];
}

int main() {
    int n;

    cout << "Enter the number of elements: ";

    cin >> n;

    vector<int> nums(n);

    cout << "Enter the elements of the set:" << endl;

    for (int i = 0; i < n; i++) {
        cin >> nums[i];
    }

    if (canPartition(nums)) {
        cout << "Output: yes" << endl;
    } else {
        cout << "Output: no" << endl;
    }
}
```

```
    return 0;  
}
```

OUTPUT

Enter the number of elements: 4

Enter the elements of the set:

1 5 11 5

Output: yes

WEEK 12

/*Q1) Given two sequences, Design an algorithm and implement it using a program to find the length of longest subsequence present in both of them. A subsequence is a sequence that appears in the same relative order, but not necessarily Contiguous*/

```
#include<iostream>

#include<bits/stdc++.h>

using namespace std;

//tabulation

int LCS(string s1, string s2) {

    int n = s1.size();

    int m = s2.size();

    vector<vector<int>>> dp(n + 1, vector<int>(m + 1, -1)); // Create a DP table

    // Initialize the base cases

    for (int i = 0; i <= n; i++) {

        dp[i][0] = 0;

    }

    for (int i = 0; i <= m; i++) {

        dp[0][i] = 0;

    }

    // Fill in the DP table to calculate the length of LCS

    for (int ind1 = 1; ind1 <= n; ind1++) {

        for (int ind2 = 1; ind2 <= m; ind2++) {

            if (s1[ind1 - 1] == s2[ind2 - 1])

                dp[ind1][ind2] = 1 + dp[ind1 - 1][ind2 - 1]; // Characters match, increment LCS length

            else

                dp[ind1][ind2] = max(dp[ind1 - 1][ind2], dp[ind1][ind2 - 1]); // Characters don't match, consider the maximum from left or above

        }

    }

    return dp[n][m]; // Return the length of the Longest Common Subsequence

}

int main()

{
```



```
int l=0;
string s1, s2;
cin>>s1>>s2;
int indx1=s1.length(), indx2=s2.length();
vector<vector<int>>>dp(indx1,vector<int>(indx2,-1));
// l=LCS(indx1-1,indx2-1,s1,s2,dp);    //for recursion and memoisation
l=LCS(s1,s2);    //TABULATION
cout<<"Longes Common Subsequence Length:"<<l;
}
```

OUTPUT

Longest Common Subsequence Length: 4

/*Q2) Given a knapsack of maximum capacity w. N items are provided, each having its own value and weight. Design an algorithm and implement it using a program to find the list of the selected items such that the final selected content has weight $\leq w$ and has maximum value. Here, you cannot break an item i.e. either pick the complete item or don't pick it. (0-1 property).*/

```
#include<iostream>

#include<bits/stdc++.h>

using namespace std;

//tabulisation

int f(int n,int W,vector<int>&val,vector<int>&wt)
{
    vector<vector<int>>dp(n,vector<int>(W,0));

    for(int i=wt[0];i<=W;i++)
        dp[0][i]=val[0];

    for(int indx=1;indx<n;indx++)
    {
        for(int w=0;w<=W;w++)
        {
            int notTake=0+dp[indx-1][W];
            int take=INT_MIN;
            if(wt[indx]<=W)
            {
                take=val[indx]+dp[indx-1][W-wt[indx]];
            }
            dp[indx][W]=max(take,notTake);
        }
    }

    return dp[n-1][W];
}

int main()
{
    int n;
    cin>>n;
```

```

int W;
cin>>W;
//Take wt array as input
vector<int> wt;
vector<vector<int>>dp(n,vector<int>(W+1,-1));
for(int i=0;i<n;i++)
{
    int v;
    cin>>v;
    wt.push_back(v);
}
//Take val indx as input
vector<int> val;
for(int i=0;i<n;i++)
{
    int v;
    cin>>v;
    val.push_back(v);
}
int maxval=f(n,W,val,wt); //for tabulation
cout<<"\nThe max val is "<<maxval<<endl;
}

```

OUTPUT

4

8

5 4 6 3

10 40 30 50

The max val is 90

/*Q3) Given a string of characters, design an algorithm and implement it using a program to print all possible permutations of the string in lexicographic order.*/

```
#include <iostream>

#include <vector>

#include <algorithm>

using namespace std;

// Function to print all permutations of a string in lexicographic order

void printPermutations(string str) {

    // Sort the string to ensure the permutations are generated in lexicographic order

    sort(str.begin(), str.end());

    // Print the permutations using next_permutation

    do {

        cout << str << endl;

    } while (next_permutation(str.begin(), str.end()));

}

int main() {

    string input;

    cout << "Enter the string: ";

    cin >> input;

    printPermutations(input);

    return 0;

}
```

OUTPUT

Enter the string: abc

abc

acb

bac

bca

cab

cba

WEEK 13

/*Q1) Given an array of characters, you have to find distinct characters from this array. Design an algorithm and implement it using a program to solve this problem using hashing. */

```
#include<iostream>

#include<bits/stdc++.h>

using namespace std;

int main()
{
    string s;
    map<char,int> freq;
    cout<<"Enter a string:"<<endl;
    cin>>s;
    for(int i=0;i<s.length();i++)
    {
        freq[s[i]]++;
    }
    cout<<"Frequency of the character of the string is"<<endl;
    for(auto it : freq)
    {
        cout<< it.first<<" "<<it.second<<endl;
    }
    return 0;
}
```

OUTPUT

Enter a string:

hello

Frequency of the character of the string is

e 1

h 1

l 2

o 1

/*Q2) Given an array of integers of size n, design an algorithm and write a program to check whether this array contains duplicate within a small window of size k < n.*/

```
#include <iostream>
```

```
#include <vector>
```

```
#include <unordered_set>
```

```
using namespace std;
```

```
void checkDuplicatesInWindow(const vector<int>& arr, int k) {
```

```
    unordered_set<int> windowElements;
```

```
    for (int i = 0; i < arr.size(); ++i) {
```

```
        if (windowElements.find(arr[i]) != windowElements.end()) {
```

```
            cout << "Duplicate present in window k" << endl;
```

```
            return;
```

```
        }
```

```
        windowElements.insert(arr[i]);
```

```
        if (windowElements.size() > k) {
```

```
            windowElements.erase(arr[i - k]);
```

```
        }
```

```
    }
```

```
    cout << "Duplicate not present in window k" << endl;
```

```
}
```

```
int main() {
```

```
    int T;
```

```
    cout << "Enter the number of test cases: ";
```

```
    cin >> T;
```

```
    while (T--) {
```

```
        int n, k;
```

```
        cout << "Enter the size of the array: ";
```

```
        cin >> n;
```

```
vector<int> arr(n);  
cout << "Enter the elements of the array: ";  
for (int i = 0; i < n; ++i) {  
    cin >> arr[i];  
}  
cout << "Enter the value of k: ";  
cin >> k;  
  
checkDuplicatesInWindow(arr, k);  
}  
return 0;  
}
```

OUTPUT

Enter the number of test cases: 1

Enter the size of the array: 6

Enter the elements of the array: 1 2 3 4 1 2

Enter the value of k: 3

Duplicate present in window k

/*Q3) Given an array of nonnegative integers, Design an algorithm and implement it using a program to find two pairs (a,b) and (c,d) such that $a*b = c*d$, where a, b, c and d are distinct elements of array.*/

```
#include <iostream>

#include <vector>

#include <unordered_map>

#include <utility> // for std::pair

using namespace std;

void findPairs(const vector<int>& arr) {

    unordered_map<int, pair<int, int>> productMap;

    int n = arr.size();

    for (int i = 0; i < n; ++i) {

        for (int j = i + 1; j < n; ++j) {

            int product = arr[i] * arr[j];

            if (productMap.find(product) != productMap.end()) {

                pair<int, int> firstPair = productMap[product];

                cout << "First pair: (" << firstPair.first << ", " << firstPair.second << ")" << endl;

                cout << "Second pair: (" << arr[i] << ", " << arr[j] << ")" << endl;

                return;

            } else {

                productMap[product] = make_pair(arr[i], arr[j]);

            }

        }

    }

    cout << "No such pairs found." << endl;

}

int main() {

    int n;

    cout << "Enter the size of the array: ";

    cin >> n;

    vector<int> arr(n);

    cout << "Enter the elements of the array: ";

    for (int i = 0; i < n; ++i) {

        cin >> arr[i];

    }

    findPairs(arr);

}
```



```
    return 0;  
}
```

OUTPUT

Enter the size of the array: 6

Enter the elements of the array: 3 4 7 1 2 9

First pair: (3, 4)

Second pair: (1, 2)

WEEK 14

/*Q1) Given a number n, write an algorithm and a program to find nth ugly number. Ugly numbers are those numbers whose only prime factors are 2, 3 or 5. The sequence 1, 2, 3, 4, 5, 6, 8, 9, 10, 12, 15, 16, 18, 20, 24,..... is sequence of ugly numbers.*/

```
#include <iostream>
```

```
using namespace std;
```

```
// Function to check if a number is an ugly number
```

```
bool isUgly(int num) {
```

```
    if (num <= 0) return false; // Non-positive numbers are not considered
```

```
    while (num % 2 == 0) num /= 2; // Divide by 2
```

```
    while (num % 3 == 0) num /= 3; // Divide by 3
```

```
    while (num % 5 == 0) num /= 5; // Divide by 5
```

```
    return num == 1; // If reduced to 1, it's an ugly number
```

```
}
```

```
int main() {
```

```
    const int n = 10; // Use a constant for array size
```

```
    int arr[n];
```

```
    cout << "Enter " << n << " elements: ";
```

```
    for (int i = 0; i < n; i++) {
```

```
        cin >> arr[i];
```

```
    }
```

```
    for (int i = 0; i < n; i++) {
```

```
        if (isUgly(arr[i])) {
```

```
            cout << arr[i] << " is an ugly number." << endl;
```

```
        }
```

```
    }
```

```
    return 0;
```

```
}
```

OUTPUT

Enter 10 elements: 12 7 6 15 9 20 25 30 18 1

12 is an ugly number.

6 is an ugly number.

15 is an ugly number.

9 is an ugly number.

20 is an ugly number.

25 is an ugly number.

30 is an ugly number.

18 is an ugly number.

1 is an ugly number.

/*Q2)Given a undriected graph ,write a algo and a prog to find the moyher vertex I a graph .A mother vertex is a vertex v such that there exist a path from v to all the other vertices of the graph.*/

```
#include <iostream>
```

```
#include <vector>
```

```
#include <list>
```

```
using namespace std;
```

```
class Graph {
```

```
    int V; // Number of vertices
```

```
    vector<list<int>> adj; // Adjacency list
```

```
    void DFSUtil(int v, vector<bool>& visited);
```

```
public:
```

```
    Graph(int V); // Constructor
```

```
    void addEdge(int v, int w); // To add an edge to the graph
```

```
    int findMother();
```

```
};
```

```
Graph::Graph(int V) {
```

```
    this->V = V;
```

```
    adj.resize(V);
```

```
}
```

```
void Graph::addEdge(int v, int w) {
```

```
    adj[v].push_back(w); // Add w to v's list.
```

```
}
```

```
void Graph::DFSUtil(int v, vector<bool>& visited) {
```

```
    visited[v] = true;
```

```
    // Recur for all the vertices adjacent to this vertex
```

```
    for (int i : adj[v]) {
```

```
        if (!visited[i]) {
```

```
            DFSUtil(i, visited);
```

```
        }
```

```

    }
}

int Graph::findMother() {
    vector<bool> visited(V, false);
    int v = 0; // To store last finished vertex

    // Perform DFS for all vertices
    for (int i = 0; i < V; i++) {
        if (!visited[i]) {
            DFSUtil(i, visited);
            v = i;
        }
    }

    // Check if the last finished vertex is a mother vertex
    fill(visited.begin(), visited.end(), false);
    DFSUtil(v, visited);

    for (int i = 0; i < V; i++) {
        if (!visited[i]) {
            return -1;
        }
    }

    return v;
}

int main() {
    int V = 7;
    Graph g(V);
    g.addEdge(0, 1);
    g.addEdge(0, 2);
    g.addEdge(1, 3);
    g.addEdge(4, 1);
    g.addEdge(6, 4);
    g.addEdge(5, 6);

```

```
g.addEdge(5, 2);
g.addEdge(6, 0);

int motherVertex = g.findMother();
if (motherVertex == -1) {
    cout << "No mother vertex found" << endl;
} else {
    cout << "The mother vertex is " << motherVertex << endl;
}

return 0;
}
```

Output

The mother vertex is 5